Chemometrics and Intelligent Laboratory Systems 231 (2022) 104685

Contents lists available at ScienceDirect

Chemometrics and Intelligent Laboratory Systems

journal homepage: www.elsevier.com/locate/chemometrics

Automatic neural network hyperparameter optimization for extrapolation: Lessons learned from visible and near-infrared spectroscopy of mango fruit

Matthew Dirks *, David Poole

University of British Columbia, 2366 Main Mall, Vancouver, British Columbia, Canada

ARTICLE INFO

Keywords:
Extrapolation
Robustness
Convolutional neural network
Ensemble averaging
Hyperparameter optimization
Automated machine learning

ABSTRACT

Configuring a neural network's architecture and hyperparameters often involves expert intuition and hand-tuning to extrapolate well without overfitting. This paper considers automatic methods for configuring a neural network for the domain of visible and near-infrared (Vis-NIR) spectroscopy. In particular, we study the effect of (a) validation set choice for validating configurations and (b) using ensembles. We consider several validation set choices: a random sample of 33% of non-test data (the technique used in previous work), samples from the latest year (a harvest season), and the first, middle, and latest 33% of samples sorted by time. To test these methods, we do a comprehensive study of a held-out 2018 harvest season of mango fruit given Vis-NIR spectra from 3 prior years. We find that ensembling improves the state-of-the-art model's variance and accuracy. Furthermore, hyperparameter optimization experiments show that when ensembling is combined with using the latest 33% of samples as the validation set, a neural network configuration is found automatically that performs as well as an expertly-chosen configuration.

1. Introduction

This paper considers how to automatically configure neural network hyperparameters such that it extrapolates for visible and near-infrared (Vis-NIR) spectroscopy. Specifically, the aim is to further streamline the process of automatic hyperparameter optimization (HPO) to produce a neural network model that generalizes, or extrapolates, well. In HPO, configurations are trained on a calibration set and evaluated on a validation set.

Two challenges in HPO are considered: (1) Overfitting to the validation set in the same way that neural network training may overfit the calibration set [1]; this is important when extrapolating (whether to new batches, across time, or beyond the range of the calibration data) [2,3]. Model updating [4] and transfer learning [5] alleviates the need for robustness toward interseason variation by fine-tuning a model using samples from a new season; this is possible when ground truth labels are available from the new season but not when doing direct extrapolation, as is considered here. (2) Variance of prediction error under different random initializations of the neural network [6,7]; this is important when datasets are small. Hence, this paper jointly considers the problem of automated hyperparameter tuning (from the field of machine learning) with the problem of extrapolation and robustness (from the field of chemometrics). To avoid overfitting, a combination of expert intuition [8], hand-tuning [9], and grid search [10] is most often used while automated methods are an open area of research [11].

For a dataset with multiple seasons, where the test set is a future season, our hypothesis is that a temporal relation exists between seasons and that validating on the latest samples will extrapolate better to future harvest seasons. To test this hypothesis, we compare several choices of partitioning the data into calibration and validation sets to use in HPO. The validation sets we test are: a random 33% (as in previous work), a semantically meaningful subset [2] (specifically, the latest season), and the first, middle, and last 33% of samples. To address the second HPO challenge, we investigate using ensembles to reduce the variance of validation-set error during hyperparameter optimization; this is tested in conjunction with partitioning choices. We conduct hyperparameter optimization for each choice of validation set and compare HPO with and without ensemble averaging. The results in this study shed light on reproducible and automated practices for configuring and training neural networks for spectroscopy in scenarios where extrapolation, to future samples or new batches, is desired.

2. Materials and methods

2.1. Data set

The dataset of mangoes by Anderson et al. [12,13] is a good example of needing to extrapolate to a new batch, in this case, a future

* Corresponding author.
E-mail addresses: mcdirks@cs.ubc.ca (M. Dirks), poole@cs.ubc.ca (D. Poole).

https://doi.org/10.1016/j.chemolab.2022.104685
Received 22 June 2022; Received in revised form 2 October 2022; Accepted 5 October 2022
Available online 10 October 2022
0169-7439/© 2022 Elsevier B.V. All rights reserved.

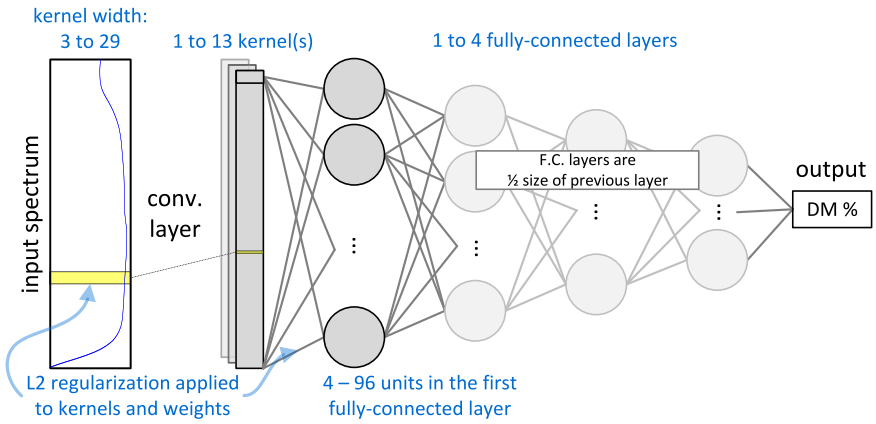

Fig. 1. Neural network architecture and hyperparameter search space. Hyperparameters are shown in blue text. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

harvest season. Visible and near-infrared (Vis-NIR) spectra of mango fruit from four harvest seasons (2015, 2016, 2017, and 2018) were collected. Using spectra from 3 years, the goal is to make predictions of samples in the 4th year. The dataset is publicly available [14]. The spectral bands range 300–1100 nm with approximately 3.3 nm intervals [12]. Near infrared spectroscopy allows for non-invasive assessment of fruit quality. In this case, the prediction target is the percent of dry matter (DM) content. DM % is an index of total carbohydrates which indicates quality of mango fruit [12].

Mishra and Passos [10] make a number of modifications to the mango fruit dataset (available online[1]), specifically: (1) only a subset (684–990 nm, 3.3 nm intervals) of the available spectral bands are used, (2) outliers have been removed from non-test-set samples, (3) chemometric pre-processing techniques were applied and concatenated together, and (4) each feature is standardized separately. Standardization of a distribution entails subtracting each value by the mean of the distribution and then dividing it by the standard deviation of the distribution. Each sample in the dataset consists of DM%, as the target to predict, and the concatenation of 6 vectors (each with 103 elements) which are:

1. The raw spectrum.
2. The first derivative of the smoothed spectrum (smoothing uses a Savitzky–Golay filter with window size of 13).
3. The second derivative of the smoothed spectrum.
4. Standardized spectrum (Standard Normal Variate, or SNV, of the spectrum).
5. The first derivative of the smoothed SNV spectrum.
6. The second derivative of the smoothed SNV spectrum.

We chose this version of the dataset because (a) it allows direct comparison to the model by Mishra and Passos [10], which is the state-of-the-art on this dataset, (b) it enables us to reproduce their model, and (c) it allows us to draw conclusions about the HPO technique without worrying about whether any observed differences were caused by a difference in pre-processing.

2.2. Baseline neural network

The state-of-the-art prediction model for this dataset is a convolutional neural network (CNN), model "B" by Mishra and Passos [10], which we will refer to as CNN_B. This model will serve as the baseline to compare our results. Its hyperparameters were optimized through a combination of multiple stages of grid search, expert intuition based on experience, and a careful analysis of overfitting. For details, readers are referred to the original paper [10] but we summarize the architecture and main hyperparameters here. The CNN_B architecture (which is similar to Fig. 1) consists of a convolutional layer (1 kernel of width 21, stride 1) followed by three fully-connected layers (of size 36, 18, and 12) with exponential linear unit (ELU) activations. Kernel and fully-connected weights are initialized by the He normal initialization method and regularized with an L_2 with a coefficient of 0.0055. Training proceeds for 750 epochs with mini-batches of size 128 and early stops when validation loss stops improving for 50 epochs. Learning rate (LR) starts at 0.005 and halves each time validation loss stops improving for 25 epochs until the minimum LR is reached (1e-6). The weights of the neural network are optimized by stochastic gradient descent using the ADAM algorithm.

2.3. Repeatability

Repeatability has been highlighted as an important issue:

> *"Differences between two runs under identical hyperparameters might still occur. Repeated experiments should be conducted to evaluate the model performance."* [9]

> *"Model repeatability and interpretation capability are critical for the further development of deep learning based spectral analysis."* [7]

Since training is stochastic (weights are initialized randomly and mini-batching randomly shuffles the data in between epochs), the weights of the network may converge to any one of many possible settings in part because there are many local optima [15]. We report the distribution of the errors given by randomly initializing and re-training, which is fairer than reporting a single sample from the distribution of errors; this improves reproducibility and reveals more about the neural network's performance [1].

2.4. Hyperparameter search space

Neural networks can take on many different architectures (such as number of layers), each with many possible hyperparameters (such as learning rate). Any specific assignment of all the hyperparameters is referred to as a configuration. In this study, the problem of neural architecture search is treated as additional hyperparameters and will be optimized in conjunction with other hyperparameters [16].

The space of possible architectures used in our hyperparameter search is based on typical architectures used in spectroscopy applications [7,9,17–22] and CNN_B [10] in particular. These neural network

[1] https://github.com/dario-passos/DeepLearning_for_VIS-NIR_Spectra/raw/master/notebooks/Tutorial_on_DL_optimization/datasets/mango_dm_full_outlier_removed2.mat

Table 1
Hyperparameters' search space.

Hyperparameter	Range
L_2 coefficient	[10^{-4}, 1]
Number of kernels	[1, 13]
Kernel width	3,5,7,11,15,21,29
Number of FC layers	[1, 4]
Number of FC units	[4, 96] (step 4)

architectures all follow a similar pattern: one or more convolutional layers followed by one or more fully-connected layers, with an activation function after each layer. Our architecture and hyperparameter search space is visualized in Fig. 1. The particular architecture and search space we use is somewhat arbitrary because the goal of the experiments is to test the HPO method under different conditions. Furthermore, we are not comparing architectures because the optimal architecture varies with data partitioning scheme and with each dataset, and specific architectures quickly become obsolete.

The search space consists of 5 hyperparameters, and we deliberately structure them to leave out conditional hyperparameters to keep it simple. More hyperparameters may also be included, with the potential of increasing prediction accuracy, but would require more computation and for the purpose of testing HPO it is not necessary. The hyperparameters and their ranges are listed in Table 1 (the ranges are based on published literature and standard practice). All fully-connected layers, other than the first one, are half the size of the previous layer; this keeps the search space smaller and simpler. For example, when there are 3 fully-connected layers and the first layer has 36 units (as in CNN_B), then the second and third layers will have 18 and 9 units. Search space for the convolution kernel's width is approximately logarithmic and rounded to the nearest odd number, ranging from 3 to 29. All hyperparameter values are integers except L_2 regularization which is a floating-point number.

2.5. Automatic hyperparameter optimization

In the field of automated machine learning (Auto-ML), the state-of-the-art for automatically searching through a hyperparameter search space is Bayesian optimization. Bayesian optimization (BO) for HPO [23–25] chooses which hyperparameter configuration to try next. The first trial configuration is sampled randomly. Subsequent configurations are selected based on a mix of how well they are estimated to perform and how uncertain the estimate is. We use an implementation of Bayesian optimization called hyperopt [26].

Pseudo-code in Fig. 2, line 6, shows the general idea: select a hyperparameter configuration (`config`) to try next via Bayesian optimization (line 7), train a neural network using that configuration (line 8), obtain a score (RMSE) for this configuration (line 9) using predictions on a validation set (validation set described in next section), and, finally, choose the configuration with the lowest RMSE (line 10). We stop HPO after completing approximately 2000 configurations (this is approximate as many were run in parallel and we stopped HPO manually once there was greater than 2000).

We follow standard practice in automatic hyperparameter optimization (HPO); readers are referred to a tutorial on HPO for spectral modeling to learn how to perform similar experiments from scratch [8]. Even when using state-of-the-art Bayesian optimization software, many decisions must be made manually, including deciding on a search space, objective function, and validation set.

2.6. Choice of validation set

Pseudo-code in Fig. 2 shows where the calibration, validation, and test sets are used in training and HPO. The test set, $\mathcal{D}^{test}$, is used in the final evaluation to score how well HPO performed (line 14). As in

```
1  function train(config, D_split)
2      NN with config, initialize randomly
3      Train NN weights on D^cal_split,
4      with ES and LR schedule based on D^val_split
5      return NN
6  function HPO(D_split)
7      for config in configurations do
8          NN = train(config, D_split)
9          RMSE_config = NN.predict(D^val_split)
10     return config with lowest RMSE
11 function final_evaluation(D_split)
12     final_config = HPO(D_split)
13     final_NN = train(best_config, D_rand)
14     RMSE = best_NN.predict(D^test)
```

Fig. 2. Experiment overview (without ensembling). Pseudo-code of neural network training (`train` function) and hyperparameter optimization (`HPO` function) showing how the sets in each partitioning choice are used and how RMSE is calculated in `HPO` and `final_evaluation`. $\mathcal{D}_{split}$ is the dataset that has been partitioned (*split*) in some way (described in Section 2.6). $\mathcal{D}^{val}_{split}$ is the validation set and $\mathcal{D}^{cal}_{split}$ is the calibration set for this *split*.

previous works [10,12,13], the 2018 harvest season is used as the test set for evaluation. For a predictive model in the fruit industry to be practical, it must be robust to year-to-year variations. Since the hope is that the model can extrapolate to a new season, a whole season is the right choice for the test set [2]. A calibration set (*cal*) is used for training the neural network weights (line 3) and a validation set (*val*) is used to inform the training procedure when to early-stop (ES) and when to drop the learning rate (LR)[2] (line 4). A validation set (*val*) is also used in HPO to score configurations (line 9).

We consider a number of ways to partition harvest seasons 2015, 2016, and 2017 (i.e. all data other than the test set) into calibration and validation sets for HPO. Note that in our experiments, the partitioning of the data ($\mathcal{D}_{split}$) will differ between training in `HPO` and training in `final_evaluation`. In previous work [10] (and standard practice), the validation set is 33% of the non-test data, randomly sampled without replacement (which we will call $\mathcal{D}^{val}_{rand}$), and the calibration set ($\mathcal{D}^{cal}_{rand}$) is the remaining 67%. Samples can also be chosen more deliberately, keeping in mind the goal to extrapolate:

1. $\mathcal{D}_{2017}$: First we select an entire season to force the model to extrapolate to a future season, just like we expect it do for the test set. Since the test set is from 2018, we use the 2017 harvest season samples (harvest seasons do not follow the calendar year but samples are labeled with their season in the dataset [14]). The 2017 harvest season samples comprise 49% of non-test data which is considerably more validation samples than before.
2. $\mathcal{D}_3$: Secondly, we partition the data to have the same size (33% in validation) as in the partitioning used in previous work, $\mathcal{D}_{rand}$. Since $\mathcal{D}_{rand}$ is used in final model training and evaluation (see line 13), matching the size may perform better because many hyperparameters are sensitive to dataset size.
3. $\mathcal{D}_1$, $\mathcal{D}_2$: To test our hypothesis that there is a temporal relation between harvest seasons, we test using the first and middle 33% of samples as well ($\mathcal{D}_1$ and $\mathcal{D}_2$ respectively). If these batches are independent, we would expect them to perform similarly.

Partitioning choices, and the sizes of each set, are summarized in Table 2. The `final_evaluation` function (in pseudo-code Fig. 2) is run

[2] The epoch at which early stopping occurs and the LR schedule (epochs at which to drop the LR) could instead be learned hyperparameters, but standard practice is to use the validation set to dynamically adapt as the training proceeds. We adopted this because we are only testing HPO with a limited set of hyperparameters.

Table 2
Partitioning choices and number of data points in validation set and corresponding calibration set. Test set is fixed with a size of 1448.

	Cal. size	Val. size	Validation set is...
$\mathcal{D}_{rand}$	6642	3272	A random 33% as in previous work [12].
$\mathcal{D}_{2017}$	5045	4869	The latest harvest season (2017).
$\mathcal{D}_1$	6642	3272	First 33% (sorted by date).
$\mathcal{D}_2$	6642	3272	Middle 33% (sorted by date).
$\mathcal{D}_3$	6642	3272	Last 33% (sorted by date).

on each partitioning choice. That is, $\mathcal{D}_{split}$ is one of $\mathcal{D}_{rand}$, $\mathcal{D}_{2017}$, $\mathcal{D}_1$, $\mathcal{D}_2$, or $\mathcal{D}_3$. Again, we emphasize, for the purpose of comparing HPO methods, regardless of which $\mathcal{D}_{split}$ is used for HPO (line 12), the final model configurations chosen in HPO **always train using** $\mathcal{D}_{rand}$ in the `final_evaluation` (line 13). Thus, any observed differences will be due to HPO, and not due to training on different data.

2.7. Neural network ensemble averaging

Neural networks are trained by minimizing an objective function, but there can be many sub-optimal weight settings after training depending on a number of factors [9] including how the weights are initialized and the order of samples seen during training. As we show in the results, a neural network's prediction score varies under different random initializations (repeating the `train` function in pseudo-code Fig. 2 results in neural networks with different weights and different prediction errors). This is problematic for HPO because the optimization is given only a weak signal to guide it. As datasets get bigger, the variance is reduced [6,7], but when that is unavailable, as is often the case, another method is to use ensembles [27].

An ensemble is when multiple models' predictions are combined to produce a final prediction. Ensembles are able to efficiently utilize these models by smoothing predictions which reduces overfitting [28]. There are different strategies for choosing models and for combining their predictions [6,27,29].

A straightforward, yet powerful, strategy is to train a neural network multiple times with different random seeds (random initializations) then average their predictions [30], which is the approach used in this paper (sometimes called ensemble averaging over initial conditions). Much more sophisticated ensemble methods exist [27], but recent literature suggests they do not live up to expectations [31]. A resurgence of recent studies, under the name Deep Ensembles [32], continues to demonstrate the effectiveness of this approach on modern deep neural networks; optimization from random initializations explores different modes leading to diverse ensemble members and, therefore, improved accuracy and out-of-distribution robustness [33].

2.8. Experiment

We conduct hyperparameter optimization (HPO) for 8 scenarios. First we compare HPO with and without ensembling using $\mathcal{D}_{rand}$, $\mathcal{D}_{2017}$, and $\mathcal{D}_3$ (6 scenarios). Then, we also test HPO on $\mathcal{D}_1$ and $\mathcal{D}_2$ but only with ensembling; this is to test whether any difference is observed as compared to $\mathcal{D}_3$. The final model configurations from each HPO scenario are compared.

2.8.1. HPO with and without ensembling

HPO without ensembling (referred to as *singles-HPO*) follows the process laid out in the pseudo-code (Fig. 2). HPO with ensembling (referred to as *ensembles-HPO*) creates an ensemble for each configuration. In Fig. 2, line 8 is repeated a number of times to make the ensemble's models. Each model in the ensemble makes predictions on the validation set, as in line 9. The predictions from the models are averaged, and RMSE for the ensemble is calculated using the averaged predictions.

Table 3
Average compute times of *ensembles-HPO* scenarios in GPU-hours. In HPO, number of models is the number of trials; in evaluation, it is the number of times the model is re-trained.

	Time per model	Number of models	Total (time per scenario)
`HPO`	0.25	~2000	533
`final_evaluation`	0.45	50	22.5

2.8.2. Final models with and without ensembling

For each HPO scenario, the final configuration from HPO (Fig. 2, line 12) is evaluated. We evaluate the baseline (CNN_B) configuration and final configurations from *singles-HPO* scenarios both as single models and as ensemble models. Since *ensembles-HPO* scenarios already utilize ensembles, we evaluate their final configurations as ensemble models only.

For single models, the final model (`final_NN`, line 13) is obtained by training on the original partitioning of the dataset ($\mathcal{D}_{rand}$). The final model makes predictions on the test set ($\mathcal{D}^{test}$). Then a distribution of RMSE scores is generated by repeatedly training and testing (repeating lines 13–14).

For ensemble models, an ensemble is created by re-training (repeating line 13), then the final RMSE is calculated using the average of the ensemble's models' predictions on the test set. A distribution of RMSE scores is again generated, this time by re-training the entire ensemble a number of times.

2.8.3. Compute hardware and software

Source code used to train the neural networks is available from https://github.com/skylogic004/spectroscopy-neural-network and requires TensorFlow 2.6 and Python 3.9. Experiments are run on a high-performance compute cluster named Cedar, hosted by Simon Fraser University for Compute Canada and Digital Research Alliance of Canada, with NVIDIA V100 and P100 GPUs and Intel Xeon E5-2650 2.2 GHz and Intel Xeon Silver 4216 2.1 GHz CPUs (1352 GPU devices and 94,528 CPU cores).

1 GPU and 6 CPUs were utilized per trial, but many were run in parallel. A total of 177.1 GPU-days were utilized in conducting the 8 GPO experiments. A user wishing to deploy HPO only has to conduct HPO once, unlike our experiments, in which case the expected compute times are listed in Table 3. A user may also reduce the number of HPO trials by stopping when satisfactory performance is achieved; for instance, we observed that 500 to 1000 trials, depending on the scenario, achieved similar validation set scores to the best score after 2000 trials.

3. Results and discussion

Experimental results are laid out in Fig. 3; we explain each panel in-turn in the following sections. First we report the distributions of single models (Sections 3.1 and 3.2, panels A and B), then we report the same model configurations but evaluated as ensembles (Sections 3.3 and 3.4, panels C and D). In Section 3.5 and panel E, we report results of HPO scenarios that use ensembles within HPO. Lastly, Section 3.8 describes a lower-bound, shown in panel F. Post-hoc analysis in Section 3.6 describes how well validation-set error correlates to test-set error. And, finally, Section 3.7 discusses the efficacy of HPO as compared to an expert. The best hyperparameter values from each HPO experiment are listed in Table 4. It is the combination of hyperparameter values that perform well; hyperparameters are not independent.

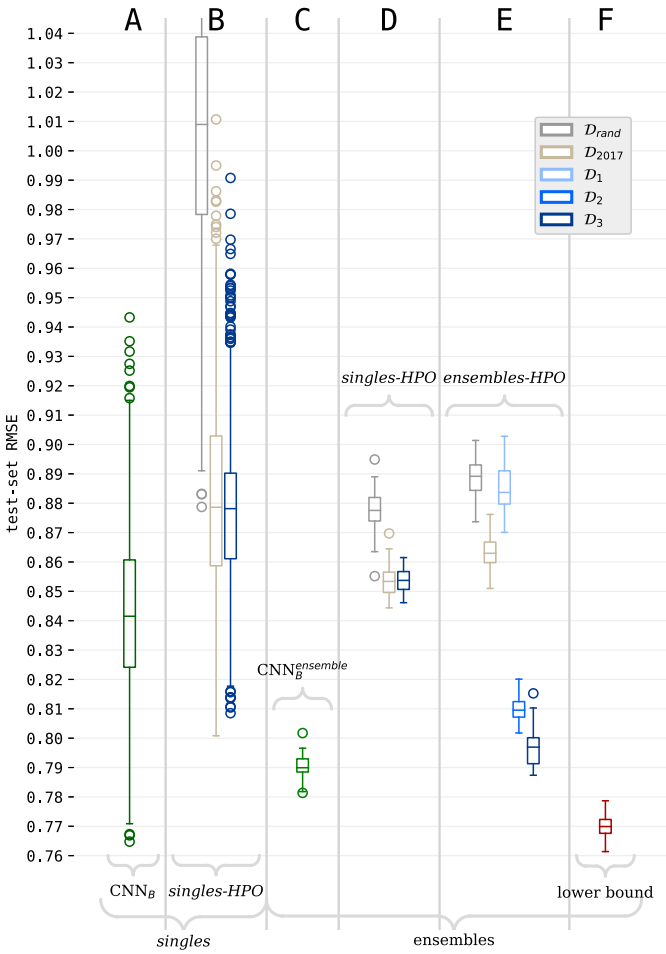

Fig. 3. Boxplot where each box a distribution over RMSE scores. The results are divided into 6 panels (labeled A–F). A is explained in Section 3.1, B in 3.2, C in 3.3, D in 3.4, E in 3.5, and F in 3.8. The top and bottom of the boxes are the upper and lower quartiles of the distribution with a line at the median. The whiskers extend from the box to show the range of the data and the points beyond the whiskers are considered outliers. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

3.1. CNN_B variance

First, we obtain the distribution of error scores of the baseline, CNN_B, by training 2000 times with random initializations. Scores are measured in RMSE, whose units match the prediction target: in this case, percent of dry matter (DM) content. The distribution is shown in Fig. 3 in panel A, it has a mean of 0.843, 95% CI [0.789, 0.896]. This means that if this model were deployed, we would expect the prediction error to be between 0.789 and 0.896 95% of the time. Mishra & Passos [10] reported 0.79 for CNN_B. One can reproduce a particular training run by seeding the random functions, but this only holds as long as nothing else changes—if you change the training data to include the test set (as you would in preparation for deployment), the resulting weights will again converge to another point in the distribution.

3.2. Singles-HPO evaluations without ensembling

Fig. 3, panel B, shows the results of the first set of hyperparameter optimization runs, *singles-HPO*, using D_{rand}, D_{2017}, and D_3. After obtaining the best neural network configuration from HPO (line 12 of Fig. 2), the neural network architecture is trained on the original (D_{rand}) partitioning of the dataset (line 13) and evaluated on the test set (line 14); lines 13–14 are repeated 2000 times to obtain a distribution of RMSE scores. The resulting distributions are visualized in the boxplot (Fig. 3, panel B).

Table 4
The best hyperparameter values found in each HPO experiment.

HPO scenario	D_{split}	L_2 coefficient	# of kernels	Kernel width	# of FC layers	# of FC units (in 1st layer)
Singles-HPO	D_{rand}	5.4391e−4	7	29	3	88
	D_{2017}	1.5369e−2	10	11	2	16
	D_3	1.7251e−2	1	7	2	32
Ensembles-HPO	D_{rand}	3.3921e−4	13	29	4	96
	D_{2017}	1.5128e−2	10	11	2	8
	D_1	1.9894e−4	4	5	2	92
	D_2	7.3074e−3	4	7	2	36
	D_3	2.9497e−3	1	15	4	16

HPO on D_{rand} was least successful (RMSE of 1.01 on average); D_3 and D_{2017} fared better with a mean around 0.88. This supports the hypothesis that HPO overfits to D_{rand}^{val} and that forcing HPO to extrapolate—by validating on samples in D_{2017}^{val} or D_3^{val}—results in a model that is better able to extrapolate to the test set. D_3 and D_{2017} HPO scenarios performed similar to each other, except D_{2017} had higher variance. Next, we look at using ensembles to reduce variance.

3.3. CNN_B evaluation with ensembling

Before doing ensemble experiments, we need to decide how many models to include in the ensemble. Since each member of the ensemble must be trained, more computation is required for each additional model, though they are readily parallelizable if parallel computing resources are available. The number of members in the ensemble can be chosen based on the trade-off between accuracy and available computation. Using the baseline, CNN_B, we compute the distribution of scores for single models ($N = 1$) up to ensembles with 500 member models ($N = 500$), shown in Fig. 4. The $N = 1$ distribution is also the distribution for CNN_B reported in Fig. 3, panel A.

The distributions in Fig. 4 are computed as follows. We first trained 2000 single models using the baseline configuration (CNN_B). Then we simulated ensembles of every size, N, from the pre-computed set of 2000. Each ensemble is a random, but unique, combination of N models. We generate 200 ensembles for each N and plot the distribution of RMSE scores.

From Fig. 4 we observe a sharp drop in variance and test-set error as soon as $N > 1$. Variance continues to decrease with greater N but tapers off. The median error also drops quickly at first, but quickly converges. We use 40 ensemble members in our experiments which balances the range of error (we deemed 0.02 from min to max as sufficient) while keeping computation reasonable based on our available compute resources. An ensemble of size 40 improves the mean error to 0.790, 95% CI [0.782, 0.799]; we refer to this model as $CNN_B^{ensemble}$. We compare the results that follow to $CNN_B^{ensemble}$, whose distribution of errors is shown in Fig. 3, panel C.

3.4. Singles-HPO evaluations with ensembling

Since ensembling CNN_B worked well, we also test the best configurations from each *singles-HPO* scenario as ensemble models. In the `final_evaluation` function (Fig. 2), the best hyperparameter configuration from HPO (line 12) is re-trained 40 times (repeat line 13 40 times). Predictions from the 40 models are averaged to make the ensemble.

Prediction error from ensembles also vary due to randomness, so we repeat the process to obtain and report the distribution of RMSE scores. Using the 2000 models from Section 3.2, we group them into groups of 40 models which makes for 50 ensembles (50 × 40 is 2000). The resulting distributions are shown in Fig. 3, panel D. The results show that the variance has tightened substantially and the error on

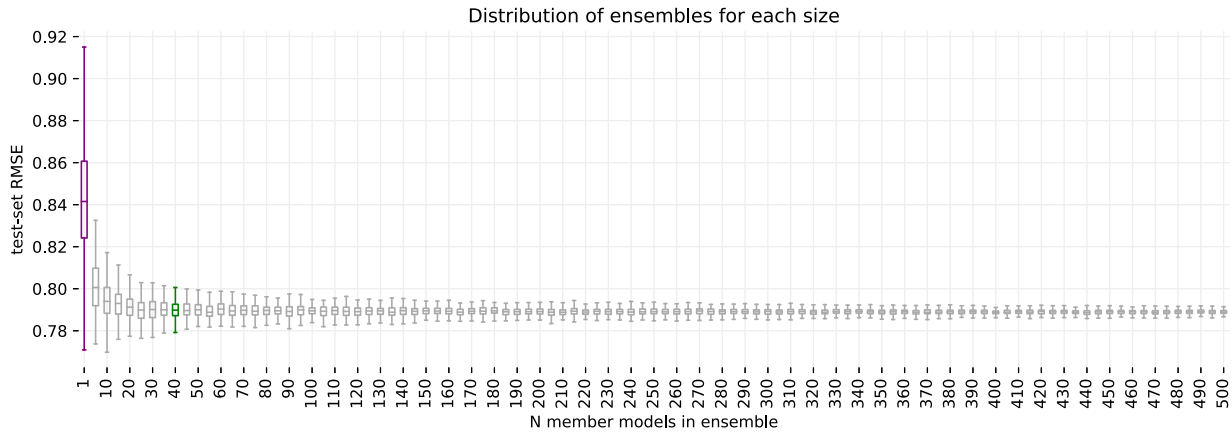

Fig. 4. RMSE of test set using CNN_B model as a function of ensemble size. Size $N = 1$ (purple) indicates the distribution of RMSE scores over 2000 single re-trainings (with random initialization) of the model. Each boxplot (where $N > 1$) is a distribution over 200 simulated ensembles, where each ensemble is formed from a unique combination of trained models (from a pre-computed set of 2000). We use ensembles with 40 member models in our experiments (green). (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

average has also dropped significantly. However, $\text{CNN}_B^{ensemble}$ remains better than all of these; the next experiment is to see if ensembling *within* hyperparameter optimization leads to an improvement.

3.5. Ensembles-HPO evaluations

The second set of hyperparameter experiments use ensembles for each configuration in HPO *and* to evaluate the final configurations. For each configuration in HPO (see `HPO` function, line 7), neural network training is repeated 40 times (repeat line 8). Then, on line 9, the ensemble's RMSE is calculated. The `final_evaluation` function proceeds with ensembling as well, as described in the previous section (Section 3.4). The resulting distributions of 50 ensemble models are visualized in the boxplot (Fig. 3, panel E).

All the partitioning choices we tested do better than $\mathcal{D}_{rand}$, but some do better than others.

Matched calibration-set size: $\mathcal{D}_3$ and $\mathcal{D}_{2017}$ are conceptually similar ($\mathcal{D}_3$ uses the latest 33% while $\mathcal{D}_{2017}$ uses the latest 49%), but $\mathcal{D}_3$'s results are significantly better. The difference in prediction error may be due to the difference in size of the calibration sets. Sizes are listed in Table 2. The size of the calibration set is important because: (a) While training networks during HPO, networks trained using $\mathcal{D}_3^{cal}$ see more data than when using $\mathcal{D}_{2017}^{cal}$, so these networks perform better on the validation set which leads HPO to a better final configuration. (b) Many hyperparameters are sensitive to dataset size, so matching the sizes between HPO and final evaluation alleviates this.

Best result: The best *ensembles-HPO* result was obtained using the $\mathcal{D}_3$ partitioning and results in a mean RMSE of 0.796, 95% CI [0.784, 0.809]. This distribution closely overlaps with the $\text{CNN}_B^{ensemble}$ distribution. This suggests that CNN_B is already very good and, indeed, Mishra & Passos [10] did an investigation into finding a model that would generalize; ensembling this model makes it even better. The combination of $\mathcal{D}_3$ with *ensembles-HPO* achieved competitive performance to $\text{CNN}_B^{ensemble}$ and this is achieved completely automatically without expert guidance.

Temporally related versus independent: If the batches (seasons) were independent, we would expect $\mathcal{D}_1$, $\mathcal{D}_2$, and $\mathcal{D}_3$ results to be similar. Instead, the results are quite different. This is evidence that the batches are not independent. Results in Fig. 3 show $\mathcal{D}_2$'s error distribution is worse than $\mathcal{D}_3$ and $\mathcal{D}_1$ is worse than $\mathcal{D}_2$. Furthermore, the hyperparameter values differ considerably between $\mathcal{D}_1$, $\mathcal{D}_2$, and $\mathcal{D}_3$ (shown in Table 4). This result may be because DM% distributions are similar between 2017 and 2018 (which is the test set) [4]. The similarity may be random or due to temporal structure. If 2017 was coincidentally very similar to 2018 (for example, in terms of climate or soil properties) then we would expect calibration sets containing 2017 season samples to lead to models that do well on the test set, and validation sets containing 2017 samples should estimate test-set error well during HPO. However:

- $\mathcal{D}_{rand}$ and $\mathcal{D}_1$ gave the two worst results, yet both contain many samples from 2017 in calibration.
- $\mathcal{D}_{rand}$ and $\mathcal{D}_{2017}$ did poorly, yet both contain many samples from 2017 in validation.

Instead we find that $\mathcal{D}_2$ and $\mathcal{D}_3$ work the best. The possibility of a temporal relation should be further tested on other datasets.

HPO with and without ensembling: Prediction error using $\mathcal{D}_3$ is greatly improved in *ensembles-HPO* as compared to *singles-HPO*, this may be explained by the reduced variance. Since ensembles reduce variance, the ensemble members can individually exhibit larger variance than single models would allow. That is, individual networks are given more leniency to overfit (or be less regularized) because the ensemble reduces the variance while keeping the error low. HPO then picks a configuration whose *ensemble* error is optimized, rather than the error of an individual model. $\mathcal{D}_{rand}$ and $\mathcal{D}_{2017}$ did not improve, however. For $\mathcal{D}_{rand}$, the reason may be that increased overfitting is detrimental given that it already exhibits overfitting. For $\mathcal{D}_{2017}$, the reason may be its smaller calibration set which also worsens overfitting. The main thing to note here is that ensembling alone is not sufficient to improve HPO; the choice of validation set also matters.

3.6. Generalizability of validation set

Hyperparameters are chosen to minimize validation error, with hopes that the hyperparameter configuration also minimizes test set error. This assumes that validation set error is correlated to test set error [1]; here we check whether this is achieved.

Fig. 5 shows validation-set and test-set error for all configurations tried during HPO and for all 8 HPO scenarios. Each point is the result of training one hyperparameter configuration on the calibration ($\mathcal{D}_{split}^{cal}$) set then evaluating it on validation ($\mathcal{D}_{split}^{val}$) and test sets. There are very few points with high validation-set or test-set error because HPO focuses its search on configurations with low validation-set error. Three important observations can be made from this figure:

1. An important observation is the "V" shape in Fig. 5(A) ($\mathcal{D}_{rand}$ with *singles-HPO* and *ensembles-HPO*); as HPO progresses toward lower validation-set error the test-set error drops at first, but then increases again. This indicates overfitting. Similar to how

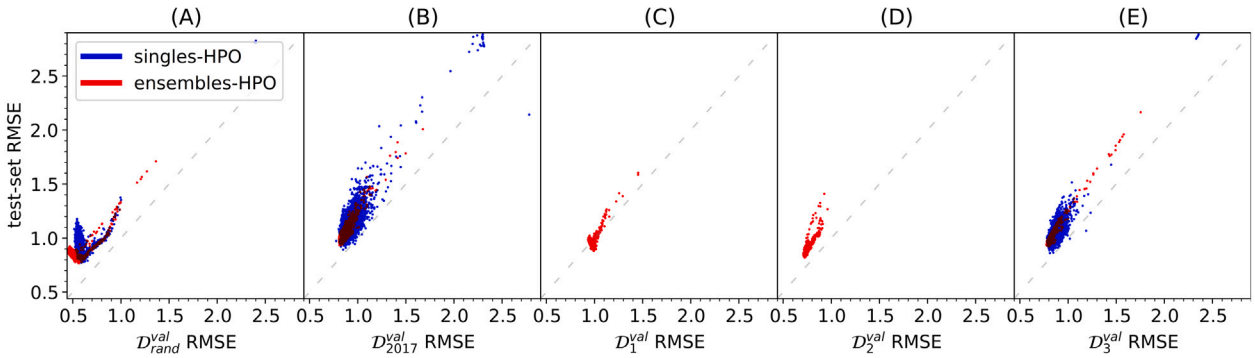

Fig. 5. Best viewed in color. Relationship between RMSE on the test set versus validation set for each HPO scenario. Blue points: HPO configurations trained once (*singles-HPO* scenarios). Red points: HPO configurations trained as an ensemble of 40 models (*ensembles-HPO* scenarios). Plot is cutoff at 2.5 on the x-axis and 2.9 on the y-axis for readability (14 points from $\mathcal{D}_{2017}$ (*singles-HPO*) are cutoff but are positioned where you had expect by linearly extrapolating to $(3.34, 4.40)$). (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

neural network weights can overfit, overfitting in model selection is a significant problem [1]. The two $\mathcal{D}_{rand}$ HPO scenarios in Fig. 5(A) exhibit overfitting likely because the validation and calibration samples are randomly partitioned which means that for every sample in the validation set, a similar sample in the calibration set can be found. Fitting the calibration set well (or overfitting it) results in a good fit of the validation set too. However, since the test set is an entire separate season, it is not as similar to the calibration samples as the validation set is, suggesting that the models are overfitting to seasonal characteristics in the data. $\mathcal{D}_1$ also exhibits the "V" shape, though to a lesser degree, so the distribution of samples in its calibration set must be too similar to its validation set, allowing for overfitting. Since the validation set for this partitioning ($\mathcal{D}^{val}_1$) contains only 2015 season samples, 2015 is not a good choice to enable extrapolation. The other partitioning choices ($\mathcal{D}_{2017}$, $\mathcal{D}_2$, and $\mathcal{D}_3$) shown in Fig. 5 appear to have a more linear and monotonic relationship between validation and test error. In these, minimizing validation-set error is a good proxy for minimizing test-set error.

2. Secondly, the cloud of points is tighter for all *ensembles-HPO* scenarios (red points in Fig. 5) than for *singles-HPO*. This shows that ensembling scenarios have better correlation between validation and test error. Furthermore, comparing $\mathcal{D}_{2017}$ to $\mathcal{D}_3$ (Fig. 5(B) and (E)) that the cloud of points is tighter for $\mathcal{D}_3$; this may be because the calibration set is smaller is $\mathcal{D}_{2017}$, which may explain why HPO worked better for $\mathcal{D}_3$.
3. Lastly, the points in $\mathcal{D}_3$ with *ensembles-HPO* appear to be the most correlated, and least spread out, across all the scenarios (red points in Fig. 5(E)), which may explain why this scenario performed the best in the final evaluation (see Fig. 3, panel E).

3.7. Baseline compared to HPO configurations

In this section we consider how the baseline (CNN_B) configuration would have performed had it been a candidate configuration during HPO. Specifically, we look at validation-set error of CNN_B compared to all the validation scores seen during two HPO scenarios: $\mathcal{D}_{rand}$ with *singles-HPO* and $\mathcal{D}_3$ with *ensembles-HPO* (the worst and best scenarios).

- CNN_B achieved an average validation-set error of 0.609 (trained using $\mathcal{D}_{rand}$). Compared to all the trials tested that use $\mathcal{D}_{rand}$ in *singles-HPO* (blue points in Fig. 5(A)), this error is approximately at the bottom of the "V" shape. This is not the minimum validation-set error, so CNN_B would not have been chosen in HPO. Rather, other configurations work better than CNN_B on the validation set. It is, however, close to where test-set error is minimized.
- When we trained $\text{CNN}^{ensemble}_B$ on $\mathcal{D}_3$ (instead of $\mathcal{D}_{rand}$) validation error is 0.792 on average. Compared to the trials that use $\mathcal{D}_3$ in *ensembles-HPO* (red points in Fig. 5(E)), this validation-set error is close to the best HPO configuration, which has a validation error of 0.774. Since HPO picked a configuration whose validation-set error is essentially on par with $\text{CNN}^{ensemble}_B$, we can say that this HPO objective is successfully guiding the search towards configurations on par with the expertly-chosen one.

3.8. Lower bound

From Fig. 5, the model configuration with the best test-set loss may give an idea of what the lower bound is for our hyperparameter search space and how far away our models are from it. The hyperparameter configuration that minimizes test-set error is from $\mathcal{D}_{rand}$ with *ensembles-HPO* (Fig. 5(A)) with a test-set error of 0.766. This configuration was not chosen by HPO, however, because its validation error was not the minimum. We cannot deploy this model because it likely overfits the test set. Re-trained 50 times, it achieves a mean of 0.770 on the test set (for reference, $\text{CNN}^{ensemble}_B$ has mean of 0.790) and its error distribution is shown in Fig. 3, panel F. This gives a lower bound on what we could possibly expect to achieve with further improvements.

4. Conclusion

The goal of this study was to elucidate hyperparameter tuning methods for building neural networks that extrapolate. Experiments showed that HPO overfit the validation set the most when the validation set was a random selection. We found that HPO worked best when trial configurations were validated on the latest 33% of samples rather than random samples. For this validation set choice, validation-set error (of HPO trials) better correlates to test-set error, thus guiding hyperparameter optimization to a model that extrapolates well. Ensembles of randomly-initialized neural networks reduced the variance and average error substantially for all HPO models and the previous state-of-the-art. Lowering the variance of the neural networks during HPO—combined with the new validation set—led to considerable improvement for hyperparameter optimization which found a hyperparameter configuration on par with $\text{CNN}^{ensemble}_B$ without requiring an expert.

For practitioners building neural networks for spectroscopy, three recommendations should be taken under consideration as a result of this study: (1) Optimize hyperparameters by validating on samples that correspond to how the model will ultimately be used. We found evidence that seasons were not independent; a validation set based on temporal structure among batches (or seasons) may lead to better HPO. (2) Reduce neural network variance and improve accuracy using ensembling, such as ensembles obtained by re-initializing the neural

network many times, re-training it, and averaging all the models' predictions. (3) Use ensembles both in deployment and in validating HPO configurations. This study used only one dataset so we cannot conclude how common it is that these recommendations apply. However, it is plausible, in this domain, that extrapolation across seasons led to the improvement. These recommendations should be considered in other domains where extrapolation may occur.

CRediT authorship contribution statement

Matthew Dirks: Conceptualization, Methodology, Software, Formal analysis, Investigation, Data curation, Writing – original draft, Visualization. **David Poole:** Conceptualization, Writing – review & editing, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

I have shared the link to my code and data (hosted on github) within the paper.

References

[1] G.C. Cawley, N.L. Talbot, On over-fitting in model selection and subsequent selection bias in performance evaluation, J. Mach. Learn. Res. 11 (Jul) (2010) 2079–2107, URL http://jmlr.csail.mit.edu/papers/v11/cawley10a.html.

[2] F. Westad, F. Marini, Validation of chemometric models – A tutorial, Anal. Chim. Acta 893 (2015) 14–24, http://dx.doi.org/10.1016/j.aca.2015.06.056, URL https://www.sciencedirect.com/science/article/pii/S0003267015008922.

[3] E.J. Bjerrum, M. Glahder, T. Skov, Data augmentation of spectral data for convolutional neural network (CNN) based deep chemometrics, 2017, CoRR abs/1710.01927, arXiv:1710.01927.

[4] J. Yang, X. Luo, X. Zhang, D. Passos, L. Xie, X. Rao, H. Xu, K. Ting, T. Lin, Y. Ying, A deep learning approach to improving spectral analysis of fruit quality under interseason variation, Food Control 140 (2022) 109108, http://dx.doi.org/10.1016/j.foodcont.2022.109108, URL https://www.sciencedirect.com/science/article/pii/S0956713522003012.

[5] P. Mishra, D. Passos, Realizing transfer learning for updating deep learning models of spectral data to be used in new scenarios, Chemometr. Intell. Lab. Syst. 212 (2021) 104283, http://dx.doi.org/10.1016/j.chemolab.2021.104283, URL https://www.sciencedirect.com/science/article/pii/S0169743921000514.

[6] T.G. Dietterich, Ensemble methods in machine learning, in: Multiple Classifier Systems, Springer Berlin Heidelberg, Berlin, Heidelberg, 2000, pp. 1–15.

[7] X. Zhang, T. Lin, J. Xu, X. Luo, Y. Ying, DeepSpectra: An end-to-end deep learning approach for quantitative spectral analysis, Anal. Chim. Acta 1058 (2019) 48–57, http://dx.doi.org/10.1016/j.aca.2019.01.002, URL http://www.sciencedirect.com/science/article/pii/S0003267019300169.

[8] D. Passos, P. Mishra, A tutorial on automatic hyperparameter tuning of deep spectral modelling for regression and classification tasks, Chemometr. Intell. Lab. Syst. 223 (2022) 104520, http://dx.doi.org/10.1016/j.chemolab.2022.104520, URL https://www.sciencedirect.com/science/article/pii/S0169743922000314.

[9] J. Yang, J. Xu, X. Zhang, C. Wu, T. Lin, Y. Ying, Deep learning for vibrational spectral analysis: Recent progress and a practical guide, Anal. Chim. Acta (2019) http://dx.doi.org/10.1016/j.aca.2019.06.012, URL http://www.sciencedirect.com/science/article/pii/S0003267019307342.

[10] P. Mishra, D. Passos, A synergistic use of chemometrics and deep learning improved the predictive performance of near-infrared spectroscopy models for dry matter prediction in mango fruit, Chemometr. Intell. Lab. Syst. 212 (2021) 104287, http://dx.doi.org/10.1016/j.chemolab.2021.104287, URL https://www.sciencedirect.com/science/article/pii/S0169743921000551.

[11] F. Hutter, L. Kotthoff, J. Vanschoren (Eds.), Automated Machine Learning - Methods, Systems, Challenges, Springer, 2019.

[12] N. Anderson, K. Walsh, P. Subedi, C. Hayes, Achieving robustness across season, location and cultivar for a NIRS model for intact mango fruit dry matter content, Postharvest Biol. Technol. 168 (2020) 111202, http://dx.doi.org/10.1016/j.postharvbio.2020.111202, URL https://www.sciencedirect.com/science/article/pii/S0925521420301629.

[13] N. Anderson, K. Walsh, J. Flynn, J. Walsh, Achieving robustness across season, location and cultivar for a NIRS model for intact mango fruit dry matter content. II. Local PLS and nonlinear models, Postharvest Biol. Technol. 171 (2021) 111358, http://dx.doi.org/10.1016/j.postharvbio.2020.111358, URL https://www.sciencedirect.com/science/article/pii/S0925521420309303.

[14] N. Anderson, Mango DMC and spectra Anderson others, 2020, 2020.

[15] L. Hansen, P. Salamon, Neural network ensembles, IEEE Trans. Pattern Anal. Mach. Intell. 12 (10) (1990) 993–1001, http://dx.doi.org/10.1109/34.58871.

[16] A. Zela, A. Klein, S. Falkner, F. Hutter, Towards automated deep learning: Efficient joint neural architecture and hyperparameter search, 2018, CoRR abs/1807.06906, arXiv:1807.06906.

[17] J. Acquarelli, T. van Laarhoven, J. Gerretzen, T.N. Tran, L.M. Buydens, E. Marchiori, Convolutional neural networks for vibrational spectroscopic data analysis, Anal. Chim. Acta 954 (2017) 22–31, http://dx.doi.org/10.1016/j.aca.2016.12.010, URL http://www.sciencedirect.com/science/article/pii/S0003267016314842.

[18] J. Liu, M. Osadchy, L. Ashton, M. Foster, C.J. Solomon, S.J. Gibson, Deep convolutional neural networks for Raman spectrum recognition: A unified solution, Analyst 142 (2017) 4067–4074, http://dx.doi.org/10.1039/C7AN01371J.

[19] S. Malek, F. Melgani, Y. Bazi, One-dimensional convolutional neural networks for spectroscopic signal regression, J. Chemometrics 32 (5) (2018) e2977, http://dx.doi.org/10.1002/cem.2977, URL https://onlinelibrary.wiley.com/doi/abs/10.1002/cem.2977, e2977 CEM-17-0106.R1.

[20] C. Cui, T. Fearn, Modern practical convolutional neural networks for multivariate regression: Applications to NIR calibration, Chemometr. Intell. Lab. Syst. 182 (2018) 9–20, http://dx.doi.org/10.1016/j.chemolab.2018.07.008.

[21] X. Fan, W. Ming, H. Zeng, Z. Zhang, H. Lu, Deep learning-based component identification for the Raman spectra of mixtures, Analyst 144 (2019) 1789–1798, http://dx.doi.org/10.1039/C8AN02212G.

[22] M. Chatzidakis, G.A. Botton, Towards calibration-invariant spectroscopy using deep learning, Sci. Rep. 9 (1) (2019) 2126, http://dx.doi.org/10.1038/s41598-019-38482-1.

[23] J. Bergstra, B. Komer, C. Eliasmith, D. Yamins, D.D. Cox, Hyperopt: A python library for model selection and hyperparameter optimization, Comput. Sci. Discov. 8 (1) (2015) 014008, http://dx.doi.org/10.1088/1749-4699/8/1/014008.

[24] B. Shahriari, K. Swersky, Z. Wang, R.P. Adams, N. de Freitas, Taking the human out of the loop: A review of Bayesian optimization, Proc. IEEE 104 (1) (2016) 148–175, http://dx.doi.org/10.1109/JPROC.2015.2494218.

[25] D. Passos, P. Mishra, An automated deep learning pipeline based on advanced optimisations for leveraging spectral classification modelling, Chemometr. Intell. Lab. Syst. 215 (2021) 104354, http://dx.doi.org/10.1016/j.chemolab.2021.104354, URL https://www.sciencedirect.com/science/article/pii/S0169743921001222.

[26] J. Bergstra, D. Yamins, D. Cox, Making a science of model search: Hyperparameter optimization in hundreds of dimensions for vision architectures, in: S. Dasgupta, D. McAllester (Eds.), Proceedings of the 30th International Conference on Machine Learning, in: Proceedings of Machine Learning Research, vol. 28, (1) PMLR, Atlanta, Georgia, USA, 2013, pp. 115–123, URL https://proceedings.mlr.press/v28/bergstra13.html.

[27] R. Polikar, Ensemble learning, in: Ensemble Machine Learning, Springer, 2012, pp. 1–34.

[28] M.P. Perrone, L.N. Cooper, When Networks Disagree: Ensemble Methods for Hybrid Neural Networks, Chapman and Hall, 1993, pp. 126–142.

[29] S. Zaidi, A. Zela, T. Elsken, C.C. Holmes, F. Hutter, Y. Teh, Neural ensemble search for uncertainty estimation and dataset shift, in: M. Ranzato, A. Beygelzimer, Y. Dauphin, P. Liang, J.W. Vaughan (Eds.), Advances in Neural Information Processing Systems, Vol. 34, Curran Associates, Inc., 2021, pp. 7898–7911, URL https://proceedings.neurips.cc/paper/2021/file/41a6fd31aa2e75c3c6d427db3d17ea80-Paper.pdf.

[30] U. Naftaly, N. Intrator, D. Horn, Optimal ensemble averaging of neural networks, Network: Comput. Neural Syst. 8 (3) (1997) 283–296, http://dx.doi.org/10.1088/0954-898X_8_3_004.

[31] A. Ashukha, A. Lyzhov, D. Molchanov, D. Vetrov, Pitfalls of in-domain uncertainty estimation and ensembling in deep learning, 2020, arXiv preprint arXiv:2002.06470.

[32] B. Lakshminarayanan, A. Pritzel, C. Blundell, Simple and scalable predictive uncertainty estimation using deep ensembles, Adv. Neural Inf. Process. Syst. 30 (2017).

[33] S. Fort, H. Hu, B. Lakshminarayanan, Deep ensembles: A loss landscape perspective, 2019, http://dx.doi.org/10.48550/ARXIV.1912.02757, arXiv, URL https://arxiv.org/abs/1912.02757.